COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	August 28, 2026	**DATE SUBMITTED:**	September 2, 2026
NAME:	Mendoza, Nathaniel B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 6.1 Searching Techniques

1. PROCEDURE OUTPUT

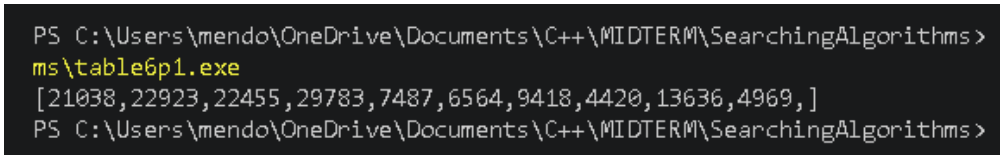
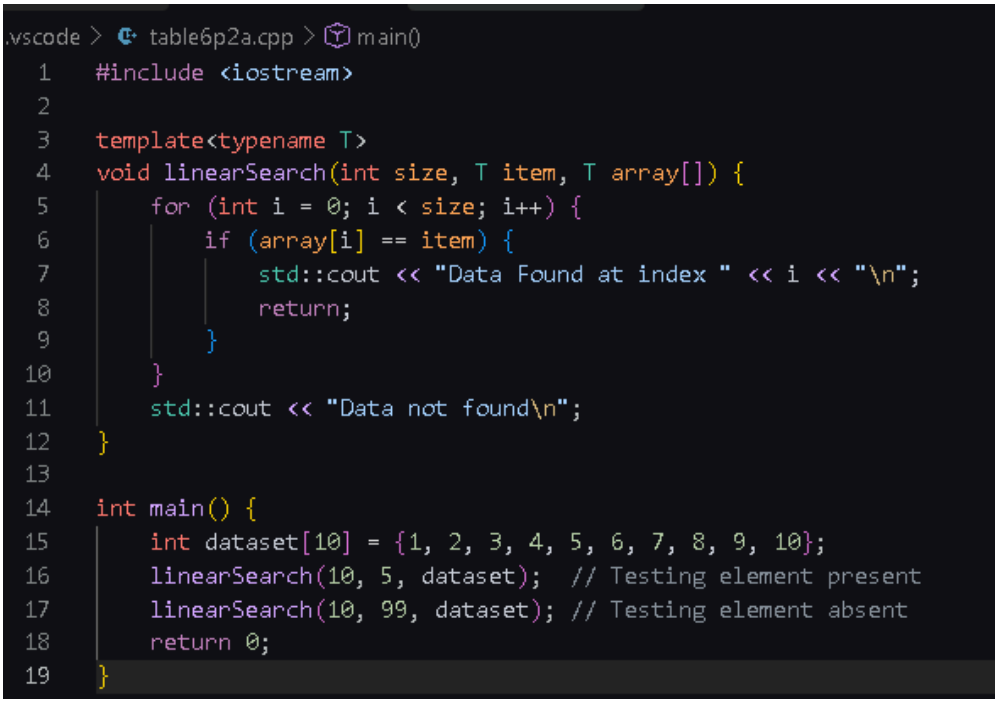
Screenshot	
Observations	This code only shows the random generated numbers wherein this only approaches the goal to fill the array using rand.

Table 6-1. Data Generated and Observations.

Code	

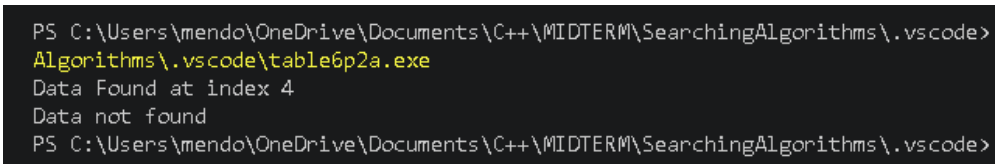
Screenshot	
Observations	This function implements a standard sequential search across our array by checking each index one by one starting from the very first element. It runs through the array and immediately returns a success message once it finds our target item, or tells us it was not found if it reaches the end.

Table 6-2a. Linear Search for Arrays

Code	table6p2b.cpp > main()
	1 #include <iostream>
	2 #include "node.h"
	3
	4 template<typename T>
	5 void linearSearchLL(Node<T> *headNode, T dataToBeFound) {
	6 while (headNode != nullptr) {
	7 if (headNode->data == dataToBeFound) {
	8 std::cout << "Found Data inside the node\n";
	9 return;
	10 }
	11 headNode = headNode->next;
	12 }
	13 std::cout << "Data not found in nodes given\n";
	14 }
	15
	16 int main() {
	17 Node<char> *name1 = newNode('R');
	18 Node<char> *name2 = newNode('o');
	19 Node<char> *name3 = newNode('m');
	20 Node<char> *name4 = newNode('a');
	21 Node<char> *name5 = newNode('n');
	22
	23 name1->next = name2;
	24 name2->next = name3;
	25 name3->next = name4;
	26 name4->next = name5;
	27
	28 linearSearchLL(name1, 'm');
	29 linearSearchLL(name1, 'z');
	30
	31 deleteNodes(&name1);
	32 return 0;
	33 }

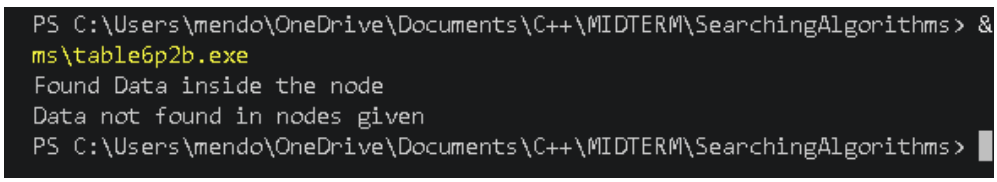
Screenshot	
Observations	This routine performs a sequential linear search adapted for linked lists by following memory pointers from node to node. It checks the data held inside the current node and moves forward to headNode->next until it either hits a match or runs out of nodes at nullptr

Table 6-2b. Linear Search for Linked List

Code	

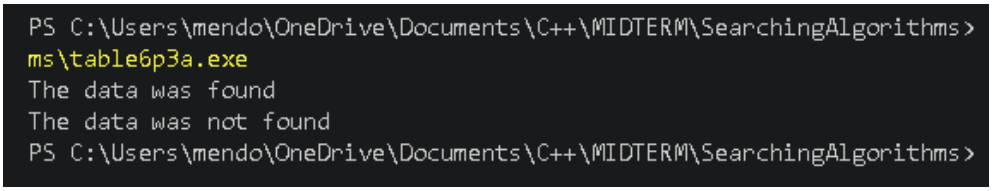
Screenshot	
Observations	This function executes binary search on a sorted array by using index math to calculate the middle element of our dataset. It continuously splits the search area in half depending on whether our target number is larger or smaller than the middle value, making the lookup process much faster.

Table 6-3a. Binary Search for Arrays

Code	table6p3b.cpp > ...
	1 #include <iostream>
	2 #include "node.h"
	3
	4 template<typename T>
	5 Node<T>* getMiddle(Node<T>* start, Node<T>* end) {
	6 Node<T>* slow = start;
	7 Node<T>* fast = start;
	8
	9 while (fast != end && fast->next != end) {
	10 slow = slow->next;
	11 fast = fast->next->next;
	12 }
	13
	14 return slow;
	15 }
	16
	17 template <typename T>
	18 void binarySearchLL(Node<T>* start, T dataFind) {
	19 Node<T>* last = nullptr;
	20
	21 while (start != last) {
	22 Node<T>* middle = getMiddle(start, last);
	23
	24 if (middle->data == dataFind) {
	25 std::cout << "Data found\n";
	26 return;
	27 }
	28 else if (middle->data > dataFind) {
	29 last = middle;
	30 }
	31 else {
	32 start = middle->next;
	33 }
	34 }
	35
	36 std::cout << "Data not found\n";
	37 }

```cpp

int main() {
    Node<int> *num1  = newNode(10);
    Node<int> *num2  = newNode(20);
    Node<int> *num3  = newNode(30);
    Node<int> *num4  = newNode(40);
    Node<int> *num5  = newNode(50);
    Node<int> *num6  = newNode(60);
    Node<int> *num7  = newNode(70);
    Node<int> *num8  = newNode(80);
    Node<int> *num9  = newNode(90);
    Node<int> *num10 = newNode(100);

    num1->next  = num2;
    num2->next  = num3;
    num3->next  = num4;
    num4->next  = num5;
    num5->next  = num6;
    num6->next  = num7;
    num7->next  = num8;
    num8->next  = num9;
    num9->next  = num10;

    binarySearchLL(num1, 66); // Testing element absent (e.g. 66)
    binarySearchLL(num1, 50); // Testing element present (e.g. 50)

    deleteNodes(&num1);
    return 0;
}
```

Screenshot

```
PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\SearchingAlgorithms>
ms\table6p3b.exe
Data not found
Data found
PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\SearchingAlgorithms>
```

Observations

This code adapts binary search for a sorted linked list by using a slow and fast pointer setup (getMiddle()) to locate the center node of our list segment. While it logically halves the search boundaries like standard binary search, traversing the list to find that middle node means the overall search time scales linearly.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

SUPPLEMENTARYNODE.H	

Code	Supplementary > SupplementaryNode.h > ...
	1 #ifndef SUPPLEMENTARYNODE_H
	2 #define SUPPLEMENTARYNODE_H
	3
	4 template<typename T>
	5 class Node {
	6 public:
	7 T data;
	8 Node* next;
	9 };
	10
	11 template<typename T>
	12 Node<T>* newNode(T newData) {
	13 Node<T>* newNode = new Node<T>;
	14 newNode->data = newData;
	15 newNode->next = nullptr;
	16 return newNode;
	17 }
	18
	19 template<typename T>
	20 void deleteNodes(Node<T>** head) {
	21 Node<T>* current = *head;
	22 while (current != nullptr) {
	23 Node<T>* nextNode = current->next;
	24 delete current;
	25 current = nextNode;
	26 }
	27 *head = nullptr;
	28 }
	29
	30 #endif

Observations	Used for creating and managing nodes for the linked list. Wherein the node class contains data for storing a value next for pointing it to the next node. While the function creates a new node and sets its next always next to nullptr. This also has deleteNode for deleting each node from memory.. This file is important because it gives the program the basic structure needed to build a linked list.

SUPPLEMENTARYSEARCHINGALGORITHM.H

Code

```cpp
#include "supplementaryNode.h"
#include <iostream>

#ifndef SUPPLEMENTARYSEARCHINGALGO_H
#define SUPPLEMENTARYSEARCHINGALGO_H

// Sequential search for Problem 1 (Array approach)
template<typename T>
void linearSearchArray(T array[], int size, T item) {
    int comparisons = 0;
    for (int i = 0; i < size; i++) {
        comparisons++;
        if (array[i] == item) {
            std::cout << "[Array] Key '" << item << "' found at index " << i
                      << " after " << comparisons << " comparison(s).\n";
            return;
        }
    }
    std::cout << "[Array] Key '" << item << "' not found after " << comparisons << " comparison(s).\n";
}

// Sequential search for Problem 1 (Linked List approach)
template<typename T>
void linearSearchLinkedList(Node<T>* head, T item) {
    int comparisons = 0;
    Node<T>* current = head;
    while (current != nullptr) {
        comparisons++;
        if (current->data == item) {
            std::cout << "[Linked List] Key '" << item << "' found after "
                      << comparisons << " comparison(s).\n";
            return;
        }
        current = current->next;
    }
    std::cout << "[Linked List] Key '" << item << "' not found after "
              << comparisons << " comparison(s).\n";
}
```

```cpp
// Problem 2: Sequential search modified to count total repeating occurrences of 'k'
template<typename T>
int countOccurrences(T array[], int size, T k) {
    int count = 0;
    for (int i = 0; i < size; i++) {
        if (array[i] == k) {
            count++;
        }
    }
    return count;
}

// Problem 3: Standard Iterative Binary Search
template<typename T>
void binarySearchIterative(T data[], int size, T dataFind) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int middle = low + (high - low) / 2;
        if (dataFind == data[middle]) {
            std::cout << "Key '" << dataFind << "' found at index " << middle << ".\n";
            return;
        }
        else if (dataFind < data[middle]) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }

    std::cout << "Key '" << dataFind << "' was not found.\n";
}
```

```cpp
// Problem 4: Recursive Binary Search
template<typename T>
void binarySearchRecursive(T data[], int low, int high, T dataFind) {
    if (low > high) {
        std::cout << "Key '" << dataFind << "' was not found.\n";
        return;
    }

    int middle = low + (high - low) / 2;

    if (dataFind == data[middle]) {
        std::cout << "Key '" << dataFind << "' found at index " << middle << ".\n";
        return;
    }
    else if (dataFind < data[middle]) {
        binarySearchRecursive(data, low, middle - 1, dataFind);
    }
    else {
        binarySearchRecursive(data, middle + 1, high, dataFind);
    }
}

#endif
```

Observations	This header file acts as a searching library containing customized linear and binary search algorithms for arrays and linked lists. The linearSearchArray and linearSearchLinkedList functions iterate sequentially through elements while tracking operation counts, confirming that finding key 18 requires exactly two comparisons on both structures. The file also includes countOccurrences to tally duplicate instances across an array, alongside binarySearchIterative and binarySearchRecursive to show how loops and call stacks perform divide-and-conquer lookups on sorted data.

PROBLEM 3 DIAGRAM

GOAL : FIND 8

Defining Numbers from Lowest to Highest to get Median (Middle Index Value)										
Index	0	1	2	3	4	5	6	7	8	9
Value	3	5	6	8	11	12	14	15	17	18

LOW

HIGH

Formula: (L.I + H.I) / 2 = Middle Index	Lowest Index= 0	Highest Index = 9	**Middle = (0+9) / 2 = Index[4] = 11**

Defining Numbers from Lowest to Highest to get Median (Middle Index Value)										
Index	0	1	2	3	4	5	6	7	8	9
Value	3	5	6	8	11	12	14	15	17	18
	LOW				MID					HIGH

BASIS : FINDING THE NUMBER THAT IS THE NEAREST AND BIGGER THAN 8				
12	>	11	>	8

Applicable Comparison		
8	<	11

Formula: (L.I + H.I) /2 = Middle Index	Lowest Index= 0	Highest Index = 3	Middle = (0+3) / 2 = Index[1] = 5

Only view index from 0-3				
Index	0	1	2	3
Value	3	5	6	8
	Low	Mid		High

Formula: (L.I + H.I) /2 = Middle Index	Lowest Index= 2	Highest Index = 3	Middle = (2+3) / 2 = Index[2] = 6

SIMPLIFY

Only view index from 2-3		
Index	2	3
Value	6	8
	L & M	High

Formula: (L.I + H.I) /2 = Middle Index	Lowest Index= 3	Highest Index = 3	Middle = (3+3) / 2 = Index[3] = 8

Last Loop	
Index	3
Value	8
	High

MATCH FOUND

EXPLANATION / PROOF

The concept behind this diagram is to track how binary search continuously narrows down the search space using index arithmetic. First, we define the lowest index (0) and highest index (9) of the array. We take their sum and divide by 2, since we are averaging the two index bounds, in order to obtain the middle index, which gives us a midpoint value of 11 at index 4. After evaluating this midpoint, we can simplify our array range. Since our goal is to find 8, and 8 is less than 11, the only portion we will focus on moving forward is index 0 to 3. We then apply the formula again by adding the new low and high bounds (0 + 3) and dividing by 2 to get the next middle index (1), which holds a value of 5. Since 8 is greater than 5, we simplify the array range again to focus on indices 2 to 3. We repeat this formula one more time (2 + 3 divided by 2) to get middle index 2, which holds value 6. Because 8 is greater than 6, we adjust our search window until index 3 is left (3 + 3 divided by 2), where the value 8 is finally reached and declared a match found.

CODE IMPLEMENTATION

```cpp
// Problem 3: Standard Iterative Binary Search
template<typename T>
void binarySearchIterative(T data[], int size, T dataFind) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int middle = low + (high - low) / 2;
        if (dataFind == data[middle]) {
            std::cout << "Key '" << dataFind << "' found at index " << middle << ".\n";
            return;
        }
        else if (dataFind < data[middle]) {
            high = middle - 1;
        }
        else {
            low = middle + 1;
        }
    }

    std::cout << "Key '" << dataFind << "' was not found.\n";
}
```

MAIN.CPP

Code

```cpp
#include <iostream>
#include "SupplementarySearchingAlgorithm.h"

int main() {
    // PROBLEM 1

    std::cout << "PROBLEM 1:\n";
    int list1[10] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};
    int key1 = 18;

    // Array Approach
    linearSearchArray(list1, 10, key1);
    // Linked List Approach
    Node<int>* head = newNode(15);
    Node<int>* curr = head;
    int remainingValues[9] = {18, 2, 19, 18, 0, 8, 14, 19, 14};
    for (int val : remainingValues) {
        curr->next = newNode(val);
        curr = curr->next;
    }
    linearSearchLinkedList(head, key1);
    deleteNodes(&head);
    std::cout << "\n";

    // PROBLEM 2
    std::cout << "PROBLEM 2:\n";
    int occurrences = countOccurrences(list1, 10, key1);
    std::cout << "Key '" << key1 << "' appears " << occurrences << " times in the list.\n\n";

    // PROBLEM 3

    std::cout << "PROBLEM 3:\n";
    int list2[10] = {3, 5, 6, 8, 11, 12, 14, 15, 17, 18};
    int key2 = 8;
    std::cout << "Iterative Binary Search for Key 8: ";
    binarySearchIterative(list2, 10, key2);
    std::cout << "\n";

    // PROBLEM 4
    std::cout << "PROBLEM 4:\n";
    std::cout << "Recursive Binary Search for Key 8: ";
    binarySearchRecursive(list2, 0, 9, key2);
    std::cout << "\n";

    return 0;
}
```

Screenshot

```
PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\SearchingAlgorithms\Supplementary>
PROBLEM 1:
[Array] Key '18' found at index 1 after 2 comparison(s).
[Linked List] Key '18' found after 2 comparison(s).

PROBLEM 2:
Key '18' appears 2 times in the list.

PROBLEM 3:
Iterative Binary Search for Key 8: Key '8' found at index 3.

PROBLEM 4:
Recursive Binary Search for Key 8: Key '8' found at index 3.

PS C:\Users\mendo\OneDrive\Documents\C++\MIDTERM\SearchingAlgorithms\Supplementary>
```

Observations	This main driver file sets up sample datasets and calls the search routines from supplementarySearchingAlgo.h. It builds the unsorted array and dynamic linked list for Problems 1 and 2, passing them to the sequential search and occurrence-counting functions. It also initializes the sorted array for Problems 3 and 4, executing both iterative and recursive binary searches for key 8 before freeing dynamic node memory using deleteNodes.

CONCLUSION

Throughout this laboratory activity, I gained a comprehensive understanding of how data structures fundamentally affect the efficiency of searching algorithms. I learned that linear search requires no preliminary sorting and operates in O(N) time on both contiguous arrays and linked lists. Conversely, binary search achieves logarithmic time complexity O(log N) on arrays due to instant index arithmetic. However, when binary search is adapted to singly linked lists, the requirement to traverse nodes sequentially inside helper functions like getMiddle() degrades its real-world performance down to linear time O(N). I feel confident in my overall performance during this activity, as I successfully implemented both iterative and recursive implementations across multiple data structures. My primary area for future improvement is refining my handling of dynamic memory management and double-pointer references when manipulating linked list structures. Additionally, I was able to gain more knowledge regarding the asymptotic notations to better understand these. As these have recently been done today, which made it more fresh and allows me to know its time & memory complexity.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: Grammarly, Google Search(Gemini) [The one that answers via after searching a question]

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar use, Explanation of concepts of binary and linear searching algorithm via Linked Lists, and Arrays.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: Implemented my own code analysis, before getting an auto-suggestion by Google

COMPUTER ENGINEERING

QUEZON CITY

Document. Asked google search the concepts needed to understand before implementing an approach.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.